JUnit & Spring Boot Testing Guide

Real-Time Order Processing System — Enterprise Microservices

> *A comprehensive, production-grade testing guide covering unit tests, integration tests, and controller tests for every layer of the application.*

Table of Contents

1. Project Architecture & Layer Analysis
2. Enterprise Testing Strategy
3. Test Annotations Reference
4. Project Test Folder Structure
5. Controller Layer Testing
6. Service Layer Testing
7. Repository Layer Testing
8. Kafka Consumer & Producer Testing
9. Security Layer Testing
10. Redis & Caching Tests
11. Mocking Guide
12. Real Production Testing Scenarios
13. Test Coverage Strategy
14. Common Testing Mistakes

---

1. Project Architecture & Layer Analysis

1.1 Microservices Overview

Microservice	Purpose	Key Layers
order-service	Order placement, cancellation, lifecycle management	Controller, Service, Repository, Kafka Producer/Consumer, Security, Exception Handler
auth-service	User registration, login, JWT token generation/validation	Controller, Service, Repository, Security (JwtUtil), Exception Handler
user-service	User profile CRUD, block/unblock functionality	Controller, Service, Repository, Kafka Producer, Security
product-service	Product catalog with Redis caching	Controller, Service, Repository, Kafka Producer, Redis Config, Security
inventory-service	Stock management with Redis distributed locking + optimistic locking	Controller, Service, Repository, Kafka Consumer/Producer, Redis Lock Config, Security
payment-service	Simulated payment processing (80% success rate)	Controller, Service, Repository, Kafka Consumer/Producer

notification-service	Notifications and audit logging via Kafka	Controller, Kafka Consumers, Repository, Security
api-gateway	JWT authentication filter, routing, circuit breaker	Gateway Filter, JwtUtil, Fallback Controller, CORS/Resilience Config
common-events	Shared Kafka event DTOs (OrderEvent, PaymentEvent, etc.)	Event POJOs only
eureka-server	Service discovery	Application class only

1.2 Layer-by-Layer Test Coverage Requirements

Layer	Test Required?	Test Type	Priority
Controller	✅ Yes	Unit (`@WebMvcTest`) + Integration	HIGH
Service	✅ Yes	Unit (Mockito)	CRITICAL
Repository	✅ Yes	Integration (`@DataJpaTest`)	MEDIUM
Kafka Consumers	✅ Yes	Unit (Mockito)	HIGH
Kafka Producers	✅ Yes	Unit (Mockito)	HIGH
Security/JWT	✅ Yes	Unit + Integration	HIGH
Exception Handler	✅ Yes	Integration via Controller tests	MEDIUM
Config Classes	⚠️ Conditional	Integration only if complex logic	LOW
DTOs/Events	❌ No	Not needed (no logic)	SKIP
Model/Entity	❌ No	Covered by repository tests	SKIP
Eureka Server	❌ No	Infrastructure — tested at deployment level	SKIP

---

2. Enterprise Testing Strategy

2.1 Why Testing is Important

- **Prevents Regression**: Code changes don't break existing functionality
- **Documents Behavior**: Tests serve as living documentation of business rules
- **Enables Refactoring**: Safe to restructure code when tests cover behavior
- **Reduces Production Bugs**: Catch issues before deployment
- **Speeds Up Development**: Fast feedback loop vs. manual testing

2.2 The Testing Pyramid

```
      /\
     /  \        E2E Tests (5%)
    /────\       - Full system flow tests
```

```
      /      \       - Slow, expensive, few
     /────────\
    /          \      Integration Tests (25%)
   /────────────\    - DB, Kafka, Redis tests
  /              \   - Moderate speed
 /────────────────\
/                  \  Unit Tests (70%)
/────────────────────\ - Service, Controller logic
                      - Fast, isolated, many
```

2.3 When to Use Each Test Type

Test Type	When to Use	What to Test	What NOT to Test
Unit Test	Always, for every service/controller method	Business logic, conditionals, exceptions, edge cases	Database queries, external services
Integration Test	For DB access, Kafka, Redis, security chains	Repository queries, entity persistence, cache behavior	Business logic (test in unit tests)
Controller Test	For every REST endpoint	HTTP status codes, request validation, response structure, error handling	Service logic (mock the service)
E2E Test	Sparingly, for critical user flows	Full order placement flow, signup → login → order	Every possible scenario

2.4 What Should NOT Be Tested

- **Getters/Setters**: Trivial code with no logic
- **DTOs/POJOs**: No business logic to validate
- **Spring Framework behavior**: Don't test that `@Autowired` works
- **Third-party libraries**: Don't test Kafka's internal message routing
- **Configuration constants**: Don't test that a port number is set correctly

---

3. Test Annotations Reference

3.1 JUnit 5 Annotations

`@Test`

Marks a method as a test case.

```java
@Test
void shouldPlaceOrderSuccessfully() {
    OrderResponse response = orderService.placeOrder(request, 1L);
    assertNotNull(response);
    assertEquals("CREATED", response.getStatus());
}
```

`@BeforeEach`

Runs before each test — use to set up common test data.

```
@BeforeEach
void setUp() {
    orderRequest = new OrderRequest();
    orderRequest.setProductId(1L);
    orderRequest.setQuantity(2);
    orderRequest.setUnitPrice(BigDecimal.valueOf(49.99));
    orderRequest.setProductName("Test Product");
}
```

`@AfterEach`

Runs after each test — use to clean up resources.

```
@AfterEach
void tearDown() {
    orderRepository.deleteAll();
}
```

`@DisplayName`

Provides a human-readable test name.

```
@Test
@DisplayName("Should return 404 when order not found")
void shouldReturn404WhenOrderNotFound() { ... }
```

`@Nested`

Groups related tests into inner classes.

```
@Nested
@DisplayName("Order Placement Tests")
class OrderPlacementTests {
    @Test void shouldPlaceOrderSuccessfully() { ... }
    @Test void shouldCalculateTotalPrice() { ... }
}

@Nested
@DisplayName("Order Cancellation Tests")
class OrderCancellationTests {
    @Test void shouldCancelOrderInCreatedStatus() { ... }
    @Test void shouldRejectCancellationOfPaidOrder() { ... }
}
```

3.2 Spring Boot Test Annotations

`@SpringBootTest`

Loads the full application context. Use for **integration tests** only.

```
@SpringBootTest
@ActiveProfile("test")
class OrderServiceIntegrationTest {
    @Autowired
    private OrderService orderService;
    // Full application context is loaded
}
```

When to use: Full integration tests, end-to-end flow tests. **When NOT to use**: Unit tests (too slow, loads entire context).

`@WebMvcTest`

Loads only the web layer (controllers + filters). Use for **controller unit tests**.

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired private MockMvc mockMvc;
    @MockBean private OrderService orderService;
    // Only web layer is loaded - fast!
}
```

When to use: Testing REST controllers in isolation. **Why**: 10x faster than `@SpringBootTest` for controller tests.

`@DataJpaTest`

Loads only JPA components (repositories + entities). Uses H2 in-memory DB by default.

```
@DataJpaTest
class OrderRepositoryTest {
    @Autowired private OrderRepository orderRepository;
    @Autowired private TestEntityManager entityManager;
    // Only JPA layer loaded with H2 database
}
```

When to use: Testing repository queries and entity persistence.

`@MockBean`

Creates a Mockito mock and registers it in the Spring context, replacing the real bean.

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @MockBean
    private OrderService orderService; // Real OrderService is replaced with mock
}
```

`@TestConfiguration`

Provides beans specifically for tests, overriding or supplementing production beans.

```
@TestConfiguration
static class TestKafkaConfig {
    @Bean
    public KafkaTemplate<String, Object> kafkaTemplate() {
        return Mockito.mock(KafkaTemplate.class);
    }
}
```

3.3 Mockito Annotations

`@Mock`

Creates a mock instance (pure Mockito, no Spring context).

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock private OrderRepository orderRepository;
    @Mock private OrderProducer orderProducer;
}
```

`@InjectMocks`

Creates the real object and injects all `@Mock` dependencies into it.

```
@InjectMocks
private OrderService orderService;
// orderRepository and orderProducer mocks are injected automatically
```

`@Captor`

Captures argument values passed to mocked methods for detailed verification.

```
@Captor private ArgumentCaptor<OrderEvent> eventCaptor;

@Test
void shouldPublishCorrectOrderEvent() {
    orderService.placeOrder(request, 1L);
    verify(orderProducer).sendOrderEvent(eventCaptor.capture());
    OrderEvent capturedEvent = eventCaptor.getValue();
    assertEquals(1L, capturedEvent.getProductId());
}
```

`@Spy`

Creates a partial mock — real methods are called unless explicitly stubbed.

```
@Spy private OrderService orderService;
// Real methods execute, but you can stub specific ones:
doReturn(mockResponse).when(orderService).getOrderById(anyLong());
```

---

4. Project Test Folder Structure

4.1 Enterprise Test Directory Layout

```
src/test/java/com/orderprocessing/orderservice/
├── controller/
│   ├── OrderControllerTest.java          ← @WebMvcTest
│   └── AdminOrderControllerTest.java     ← @WebMvcTest
├── service/
│   └── OrderServiceTest.java             ← @ExtendWith(MockitoExtension)
├── repository/
│   └── OrderRepositoryTest.java          ← @DataJpaTest
├── kafka/
│   ├── OrderProducerTest.java            ← @ExtendWith(MockitoExtension)
│   ├── InventoryStatusConsumerTest.java   ← @ExtendWith(MockitoExtension)
│   └── OrderStatusConsumerTest.java       ← @ExtendWith(MockitoExtension)
├── config/
│   └── SecurityConfigTest.java           ← @SpringBootTest (if needed)
├── exception/
│   └── GlobalExceptionHandlerTest.java   ← Tested via controller tests
└── integration/
    └── OrderFlowIntegrationTest.java     ← @SpringBootTest
```

4.2 Why Mirror the Main Structure?

- **Discoverability**: Developers instantly know where to find tests for any class
- **Convention over configuration**: Maven/Gradle auto-discover tests matching package structure
- **Code review efficiency**: Reviewers can quickly verify test coverage for changed files
- **IDE navigation**: Jump between source and test with one click (Ctrl+Shift+T in IntelliJ)

---

5. Controller Layer Testing

5.1 OrderController Test — Complete Example

```
package com.orderprocessing.orderservice.controller;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.orderprocessing.orderservice.dto.OrderRequest;
import com.orderprocessing.orderservice.dto.OrderResponse;
import com.orderprocessing.orderservice.service.OrderService;
import org.junit.jupiter.api.*;
import org.springframework.beans.factory.annotation.Autowired;
```

```java
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.bean.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.Arrays;
import java.util.List;

import static org.hamcrest.Matchers.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.anyLong;
import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(OrderController.class)
@DisplayName("OrderController Tests")
class OrderControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @MockBean
    private OrderService orderService;

    private OrderRequest validRequest;
    private OrderResponse sampleResponse;

    @BeforeEach
    void setUp() {
        validRequest = new OrderRequest();
        validRequest.setProductId(1L);
        validRequest.setProductName("Laptop");
        validRequest.setQuantity(2);
        validRequest.setUnitPrice(BigDecimal.valueOf(999.99));

        sampleResponse = new OrderResponse();
        sampleResponse.setId(1L);
        sampleResponse.setUserId(1L);
        sampleResponse.setProductId(1L);
        sampleResponse.setProductName("Laptop");
        sampleResponse.setQuantity(2);
        sampleResponse.setTotalPrice(BigDecimal.valueOf(1999.98));
        sampleResponse.setStatus("CREATED");
        sampleResponse.setCreatedAt(LocalDateTime.now());
    }
```

```java
// ── Success Scenarios ──────────────────────────────────

@Nested
@DisplayName("POST /api/orders — Place Order")
class PlaceOrderTests {

    @Test
    @DisplayName("Should place order and return 201 CREATED")
    void shouldPlaceOrderSuccessfully() throws Exception {
        when(orderService.placeOrder(any(OrderRequest.class), anyLong()))
            .thenReturn(sampleResponse);

        mockMvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(validRequest))
                .header("X-User-Name", "testuser"))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.status").value("CREATED"))
            .andExpect(jsonPath("$.productName").value("Laptop"))
            .andExpect(jsonPath("$.totalPrice").value(1999.98));

        verify(orderService, times(1)).placeOrder(any(), anyLong());
    }

    @Test
    @DisplayName("Should return 400 when productId is null")
    void shouldReturn400WhenProductIdIsNull() throws Exception {
        OrderRequest invalidRequest = new OrderRequest();
        invalidRequest.setQuantity(1);
        invalidRequest.setUnitPrice(BigDecimal.TEN);
        // productId is null — @NotNull violation

        mockMvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(invalidRequest)))
            .andExpect(status().isBadRequest());

        verify(orderService, never()).placeOrder(any(), anyLong());
    }

    @Test
    @DisplayName("Should return 400 when quantity is zero")
    void shouldReturn400WhenQuantityIsZero() throws Exception {
        validRequest.setQuantity(0); // @Min(1) violation

        mockMvc.perform(post("/api/orders")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(validRequest)))
            .andExpect(status().isBadRequest());
    }
}
```

```java
// ── GET Endpoints ──────────────────────────────────────────

@Nested
@DisplayName("GET /api/orders — Retrieve Orders")
class GetOrderTests {

    @Test
    @DisplayName("Should return order by ID")
    void shouldReturnOrderById() throws Exception {
        when(orderService.getOrderById(1L)).thenReturn(sampleResponse);

        mockMvc.perform(get("/api/orders/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.productName").value("Laptop"));
    }

    @Test
    @DisplayName("Should return all orders")
    void shouldReturnAllOrders() throws Exception {
        when(orderService.getAllOrders())
            .thenReturn(List.of(sampleResponse));

        mockMvc.perform(get("/api/orders"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(1)))
            .andExpect(jsonPath("$[0].id").value(1));
    }

    @Test
    @DisplayName("Should return orders by userId")
    void shouldReturnOrdersByUserId() throws Exception {
        when(orderService.getOrdersByUserId(1L))
            .thenReturn(List.of(sampleResponse));

        mockMvc.perform(get("/api/orders/user/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$", hasSize(1)));
    }
}

// ── Cancel Order ───────────────────────────────────────────

@Nested
@DisplayName("PUT /api/orders/{id}/cancel — Cancel Order")
class CancelOrderTests {

    @Test
    @DisplayName("Should cancel order successfully")
    void shouldCancelOrder() throws Exception {
        sampleResponse.setStatus("CANCELLED");
```

```
            when(orderService.cancelOrder(1L, 1L)).thenReturn(sampleResponse);

            mockMvc.perform(put("/api/orders/1/cancel")
                    .header("X-User-Name", "testuser"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.status").value("CANCELLED"));
        }

        @Test
        @DisplayName("Should return 400 when cancelling non-existent order")
        void shouldReturn400WhenOrderNotFound() throws Exception {
            when(orderService.cancelOrder(999L, 1L))
                .thenThrow(new RuntimeException("Order not found with id: 999"));

            mockMvc.perform(put("/api/orders/999/cancel")
                    .header("X-User-Name", "testuser"))
                .andExpect(status().isBadRequest());
        }
    }
}
```

5.2 AuthController Test

```
@WebMvcTest(AuthController.class)
@DisplayName("AuthController Tests")
class AuthControllerTest {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @MockBean private AuthService authService;

    @Test
    @DisplayName("POST /api/auth/signup — Should register user")
    void shouldRegisterUser() throws Exception {
        SignupRequest request = new SignupRequest();
        request.setUsername("newuser");
        request.setEmail("new@test.com");
        request.setPassword("Password123!");

        AuthResponse response = new AuthResponse(null, "newuser", "USER",
            "User registered successfully");
        when(authService.signup(any())).thenReturn(response);

        mockMvc.perform(post("/api/auth/signup")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.username").value("newuser"))
            .andExpect(jsonPath("$.message").value("User registered successfully"));
    }
```

```java
    @Test
    @DisplayName("POST /api/auth/login — Should return JWT token")
    void shouldLoginAndReturnToken() throws Exception {
        AuthRequest request = new AuthRequest();
        request.setUsername("admin");
        request.setPassword("admin123");

        AuthResponse response = new AuthResponse("jwt.token.here",
            "admin", "ADMIN", "Login successful");
        when(authService.login(any())).thenReturn(response);

        mockMvc.perform(post("/api/auth/login")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.token").value("jwt.token.here"))
            .andExpect(jsonPath("$.role").value("ADMIN"));
    }

    @Test
    @DisplayName("POST /api/auth/login — Should return error for wrong password")
    void shouldFailLoginWithWrongPassword() throws Exception {
        AuthRequest request = new AuthRequest();
        request.setUsername("admin");
        request.setPassword("wrongpass");

        when(authService.login(any()))
            .thenThrow(new RuntimeException("Invalid credentials"));

        mockMvc.perform(post("/api/auth/login")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest());
    }

    @Test
    @DisplayName("GET /api/auth/validate — Should validate token")
    void shouldValidateToken() throws Exception {
        when(authService.validateToken("valid.token")).thenReturn(true);

        mockMvc.perform(get("/api/auth/validate")
                .param("token", "valid.token"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.valid").value(true));
    }
}
```

---

6. Service Layer Testing

6.1 OrderService Test — Complete Example

```java
package com.orderprocessing.orderservice.service;

import com.orderprocessing.events.OrderEvent;
import com.orderprocessing.orderservice.dto.OrderRequest;
import com.orderprocessing.orderservice.dto.OrderResponse;
import com.orderprocessing.orderservice.kafka.OrderProducer;
import com.orderprocessing.orderservice.model.Order;
import com.orderprocessing.orderservice.repository.OrderRepository;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
@DisplayName("OrderService Unit Tests")
class OrderServiceTest {

    @Mock private OrderRepository orderRepository;
    @Mock private OrderProducer orderProducer;
    @InjectMocks private OrderService orderService;
    @Captor private ArgumentCaptor<OrderEvent> eventCaptor;
    @Captor private ArgumentCaptor<Order> orderCaptor;

    private OrderRequest orderRequest;
    private Order savedOrder;

    @BeforeEach
    void setUp() {
        orderRequest = new OrderRequest();
        orderRequest.setProductId(1L);
        orderRequest.setProductName("Laptop");
        orderRequest.setQuantity(2);
        orderRequest.setUnitPrice(BigDecimal.valueOf(999.99));

        savedOrder = new Order();
        savedOrder.setId(1L);
        savedOrder.setUserId(1L);
        savedOrder.setProductId(1L);
        savedOrder.setProductName("Laptop");
        savedOrder.setQuantity(2);
```

```java
        savedOrder.setTotalPrice(BigDecimal.valueOf(1999.98));
        savedOrder.setStatus(Order.OrderStatus.CREATED);
    }

    // ── Place Order Tests ──────────────────────────────────

    @Nested
    @DisplayName("placeOrder()")
    class PlaceOrderTests {

        @Test
        @DisplayName("Should save order with CREATED status")
        void shouldSaveOrderWithCreatedStatus() {
            when(orderRepository.save(any(Order.class))).thenReturn(savedOrder);
            doNothing().when(orderProducer).sendOrderEvent(any());

            OrderResponse response = orderService.placeOrder(orderRequest, 1L);

            assertNotNull(response);
            assertEquals("CREATED", response.getStatus());
            assertEquals(1L, response.getProductId());
            verify(orderRepository).save(orderCaptor.capture());
            assertEquals(Order.OrderStatus.CREATED, orderCaptor.getValue().getStatus());
        }

        @Test
        @DisplayName("Should calculate total price correctly")
        void shouldCalculateTotalPriceCorrectly() {
            when(orderRepository.save(any(Order.class))).thenReturn(savedOrder);

            orderService.placeOrder(orderRequest, 1L);

            verify(orderRepository).save(orderCaptor.capture());
            Order captured = orderCaptor.getValue();
            // 999.99 * 2 = 1999.98
            assertEquals(BigDecimal.valueOf(1999.98), captured.getTotalPrice());
        }

        @Test
        @DisplayName("Should publish Kafka event with correct data")
        void shouldPublishKafkaEvent() {
            when(orderRepository.save(any(Order.class))).thenReturn(savedOrder);

            orderService.placeOrder(orderRequest, 1L);

            verify(orderProducer).sendOrderEvent(eventCaptor.capture());
            OrderEvent event = eventCaptor.getValue();
            assertEquals(1L, event.getOrderId());
            assertEquals(1L, event.getProductId());
            assertEquals(2, event.getQuantity());
            assertEquals("CREATED", event.getStatus());
        }
```

```java
}

// ── Cancel Order Tests ──────────────────────────────────

@Nested
@DisplayName("cancelOrder()")
class CancelOrderTests {

    @Test
    @DisplayName("Should cancel order in CREATED status")
    void shouldCancelCreatedOrder() {
        savedOrder.setStatus(Order.OrderStatus.CREATED);
        when(orderRepository.findById(1L)).thenReturn(Optional.of(savedOrder));
        when(orderRepository.save(any())).thenReturn(savedOrder);

        OrderResponse response = orderService.cancelOrder(1L, 1L);

        assertEquals("CANCELLED", response.getStatus());
        verify(orderProducer).sendOrderCancelledEvent(any());
    }

    @Test
    @DisplayName("Should throw exception when cancelling paid order")
    void shouldThrowWhenCancellingPaidOrder() {
        savedOrder.setStatus(Order.OrderStatus.PAYMENT_SUCCESS);
        when(orderRepository.findById(1L)).thenReturn(Optional.of(savedOrder));

        RuntimeException ex = assertThrows(RuntimeException.class,
            () -> orderService.cancelOrder(1L, 1L));
        assertTrue(ex.getMessage().contains("cannot be cancelled"));
        verify(orderProducer, never()).sendOrderCancelledEvent(any());
    }

    @Test
    @DisplayName("Should throw when order not found")
    void shouldThrowWhenOrderNotFound() {
        when(orderRepository.findById(999L)).thenReturn(Optional.empty());

        assertThrows(RuntimeException.class,
            () -> orderService.cancelOrder(999L, 1L));
    }

    @Test
    @DisplayName("Should throw when user cancels another user's order")
    void shouldThrowWhenNotOwner() {
        savedOrder.setUserId(1L);
        when(orderRepository.findById(1L)).thenReturn(Optional.of(savedOrder));

        assertThrows(RuntimeException.class,
            () -> orderService.cancelOrder(1L, 999L)); // Different userId
    }
}
```

```java
    // ── Query Tests ──────────────────────────────────────────

    @Nested
    @DisplayName("Query Methods")
    class QueryTests {

        @Test
        @DisplayName("Should return order by ID")
        void shouldReturnOrderById() {
            when(orderRepository.findById(1L)).thenReturn(Optional.of(savedOrder));

            OrderResponse response = orderService.getOrderById(1L);

            assertNotNull(response);
            assertEquals(1L, response.getId());
        }

        @Test
        @DisplayName("Should return orders by userId")
        void shouldReturnOrdersByUserId() {
            when(orderRepository.findByUserIdOrderByCreatedAtDesc(1L))
                .thenReturn(List.of(savedOrder));

            List<OrderResponse> responses = orderService.getOrdersByUserId(1L);

            assertEquals(1, responses.size());
        }
    }
}
```

6.2 AuthService Test

```java
@ExtendWith(MockitoExtension.class)
@DisplayName("AuthService Unit Tests")
class AuthServiceTest {

    @Mock private UserCredentialRepository repository;
    @Mock private PasswordEncoder passwordEncoder;
    @Mock private JwtUtil jwtUtil;
    @InjectMocks private AuthService authService;

    @Test
    @DisplayName("Should register new user successfully")
    void shouldRegisterNewUser() {
        SignupRequest request = new SignupRequest();
        request.setUsername("newuser");
        request.setEmail("new@test.com");
        request.setPassword("Password123!");
```

```java
    when(repository.existsByUsername("newuser")).thenReturn(false);
    when(repository.existsByEmail("new@test.com")).thenReturn(false);
    when(passwordEncoder.encode("Password123!")).thenReturn("$2a$encoded");

    AuthResponse response = authService.signup(request);

    assertEquals("newuser", response.getUsername());
    verify(repository).save(any(UserCredential.class));
}

@Test
@DisplayName("Should throw when username already exists")
void shouldThrowWhenDuplicateUsername() {
    SignupRequest request = new SignupRequest();
    request.setUsername("existing");
    when(repository.existsByUsername("existing")).thenReturn(true);

    assertThrows(RuntimeException.class, () -> authService.signup(request));
    verify(repository, never()).save(any());
}

@Test
@DisplayName("Should login and return JWT token")
void shouldLoginSuccessfully() {
    AuthRequest request = new AuthRequest();
    request.setUsername("admin");
    request.setPassword("admin123");

    UserCredential user = new UserCredential("admin", "a@test.com",
        "$2a$encoded", UserCredential.Role.ADMIN);
    when(repository.findByUsername("admin")).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("admin123", "$2a$encoded")).thenReturn(true);
    when(jwtUtil.generateToken("admin", "ADMIN")).thenReturn("jwt.token");

    AuthResponse response = authService.login(request);

    assertEquals("jwt.token", response.getToken());
    assertEquals("ADMIN", response.getRole());
}

@Test
@DisplayName("Should throw on invalid password")
void shouldThrowOnInvalidPassword() {
    AuthRequest request = new AuthRequest();
    request.setUsername("admin");
    request.setPassword("wrong");

    UserCredential user = new UserCredential("admin", "a@test.com",
        "$2a$encoded", UserCredential.Role.ADMIN);
    when(repository.findByUsername("admin")).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("wrong", "$2a$encoded")).thenReturn(false);
```

```
        assertThrows(RuntimeException.class, () -> authService.login(request));
    }
}
```

6.3 PaymentService Test

```
@ExtendWith(MockitoExtension.class)
@DisplayName("PaymentService Unit Tests")
class PaymentServiceTest {

    @Mock private PaymentRepository repository;
    @InjectMocks private PaymentService paymentService;

    @Test
    @DisplayName("Should process payment and save to repository")
    void shouldProcessPayment() {
        when(repository.save(any(Payment.class))).thenAnswer(inv -> {
            Payment p = inv.getArgument(0);
            p.setId(1L);
            return p;
        });

        Payment result = paymentService.processPayment(1L, BigDecimal.valueOf(99.99));

        assertNotNull(result);
        assertEquals(1L, result.getOrderId());
        assertNotNull(result.getTransactionId());
        assertTrue(result.getStatus() == Payment.PaymentStatus.SUCCESS
            || result.getStatus() == Payment.PaymentStatus.FAILED);
        verify(repository).save(any());
    }
}
```

---

7. Repository Layer Testing

7.1 OrderRepository Test

```
package com.orderprocessing.orderservice.repository;

import com.orderprocessing.orderservice.model.Order;
import org.junit.jupiter.api.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;
import org.springframework.test.context.ActiveProfiles;

import java.math.BigDecimal;
```

```java
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
@ActiveProfiles("test")
@DisplayName("OrderRepository Tests")
class OrderRepositoryTest {

    @Autowired private OrderRepository orderRepository;
    @Autowired private TestEntityManager entityManager;

    private Order createOrder(Long userId, String status) {
        Order order = new Order();
        order.setUserId(userId);
        order.setProductId(1L);
        order.setProductName("Test Product");
        order.setQuantity(1);
        order.setTotalPrice(BigDecimal.valueOf(49.99));
        order.setStatus(Order.OrderStatus.valueOf(status));
        return entityManager.persistAndFlush(order);
    }

    @Test
    @DisplayName("Should find orders by userId ordered by createdAt DESC")
    void shouldFindOrdersByUserIdOrderByCreatedAtDesc() {
        createOrder(1L, "CREATED");
        createOrder(1L, "COMPLETED");
        createOrder(2L, "CREATED");

        List<Order> orders = orderRepository.findByUserIdOrderByCreatedAtDesc(1L);

        assertEquals(2, orders.size());
        orders.forEach(o -> assertEquals(1L, o.getUserId()));
    }

    @Test
    @DisplayName("Should return empty list for non-existent userId")
    void shouldReturnEmptyForNonExistentUser() {
        List<Order> orders = orderRepository.findByUserIdOrderByCreatedAtDesc(999L);
        assertTrue(orders.isEmpty());
    }

    @Test
    @DisplayName("Should persist order with all fields")
    void shouldPersistOrderWithAllFields() {
        Order order = createOrder(1L, "CREATED");
        Order found = orderRepository.findById(order.getId()).orElseThrow();

        assertEquals(1L, found.getUserId());
        assertEquals("Test Product", found.getProductName());
        assertEquals(Order.OrderStatus.CREATED, found.getStatus());
```

```
    }
}
```

7.2 Test application.yml (H2 Configuration)

```yaml
# src/test/resources/application-test.yml
spring:
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
    database-platform: org.hibernate.dialect.H2Dialect
  kafka:
    bootstrap-servers: localhost:9092  # Will be mocked in tests
```

---

8. Kafka Consumer & Producer Testing

8.1 OrderProducer Test

```java
@ExtendWith(MockitoExtension.class)
@DisplayName("OrderProducer Tests")
class OrderProducerTest {

    @Mock private KafkaTemplate<String, Object> kafkaTemplate;
    @InjectMocks private OrderProducer orderProducer;

    @Captor private ArgumentCaptor<String> topicCaptor;
    @Captor private ArgumentCaptor<String> keyCaptor;
    @Captor private ArgumentCaptor<Object> valueCaptor;

    @Test
    @DisplayName("Should send order event to 'order-created' topic with productId as key")
    void shouldSendOrderEvent() {
        OrderEvent event = new OrderEvent("evt-1", 1L, 1L, 42L,
            "Laptop", 2, BigDecimal.valueOf(1999.98), "CREATED");

        CompletableFuture<SendResult<String, Object>> future =
            CompletableFuture.completedFuture(null);
        when(kafkaTemplate.send(anyString(), anyString(), any())).thenReturn(future);

        orderProducer.sendOrderEvent(event);

        verify(kafkaTemplate).send(topicCaptor.capture(),
```

```
            keyCaptor.capture(), valueCaptor.capture());
        assertEquals("order-created", topicCaptor.getValue());
        assertEquals("42", keyCaptor.getValue()); // productId as key
    }

    @Test
    @DisplayName("Should send cancel event to 'order-cancelled' topic")
    void shouldSendCancelledEvent() {
        OrderEvent event = new OrderEvent("evt-2", 1L, 1L, 42L,
            "Laptop", 2, BigDecimal.valueOf(1999.98), "CANCELLED");

        CompletableFuture<SendResult<String, Object>> future =
            CompletableFuture.completedFuture(null);
        when(kafkaTemplate.send(anyString(), anyString(), any())).thenReturn(future);

        orderProducer.sendOrderCancelledEvent(event);

        verify(kafkaTemplate).send(eq("order-cancelled"), anyString(), eq(event));
    }
}
```

8.2 InventoryStatusConsumer Test

```
@ExtendWith(MockitoExtension.class)
@DisplayName("InventoryStatusConsumer Tests")
class InventoryStatusConsumerTest {

    @Mock private OrderRepository orderRepository;
    @InjectMocks private InventoryStatusConsumer consumer;

    @Test
    @DisplayName("Should update order to INVENTORY_RESERVED on RESERVED event")
    void shouldUpdateToInventoryReserved() {
        Order order = new Order();
        order.setId(1L);
        order.setStatus(Order.OrderStatus.CREATED);
        when(orderRepository.findById(1L)).thenReturn(Optional.of(order));

        InventoryEvent event = new InventoryEvent(
            "evt-1", 1L, 1L, 42L, 2, 100, "RESERVED");

        consumer.handleInventoryUpdated(event);

        verify(orderRepository).save(argThat(savedOrder ->
            savedOrder.getStatus() == Order.OrderStatus.INVENTORY_RESERVED));
    }

    @Test
    @DisplayName("Should cancel order on INSUFFICIENT_STOCK event")
    void shouldCancelOnInsufficientStock() {
```

```java
        Order order = new Order();
        order.setId(1L);
        order.setStatus(Order.OrderStatus.CREATED);
        when(orderRepository.findById(1L)).thenReturn(Optional.of(order));

        InventoryEvent event = new InventoryEvent(
            "evt-2", 1L, 1L, 42L, 2, 0, "INSUFFICIENT_STOCK");

        consumer.handleInventoryUpdated(event);

        verify(orderRepository).save(argThat(savedOrder ->
            savedOrder.getStatus() == Order.OrderStatus.CANCELLED));
    }

    @Test
    @DisplayName("Should not update on RELEASED event (already handled)")
    void shouldNotUpdateOnReleasedEvent() {
        InventoryEvent event = new InventoryEvent(
            "evt-3", 1L, 1L, 42L, 2, 100, "RELEASED");

        consumer.handleInventoryUpdated(event);

        verify(orderRepository, never()).save(any());
    }
}
```

8.3 PaymentConsumer Test

```java
@ExtendWith(MockitoExtension.class)
@DisplayName("PaymentConsumer Tests")
class PaymentConsumerTest {

    @Mock private PaymentService paymentService;
    @Mock private PaymentProducer paymentProducer;
    @InjectMocks private PaymentConsumer paymentConsumer;

    @Test
    @DisplayName("Should process payment when inventory is RESERVED")
    void shouldProcessPaymentOnReservedInventory() {
        Payment payment = new Payment();
        payment.setStatus(Payment.PaymentStatus.SUCCESS);
        payment.setTransactionId("txn-123");
        payment.setAmount(BigDecimal.valueOf(99.99));

        when(paymentService.processPayment(anyLong(), any())).thenReturn(payment);

        InventoryEvent event = new InventoryEvent(
            "evt-1", 1L, 1L, 42L, 2, 100, "RESERVED");

        paymentConsumer.handleInventoryUpdated(event);
```

```
        verify(paymentService).processPayment(1L, BigDecimal.valueOf(99.99));
        verify(paymentProducer).sendPaymentEvent(eq("payment-success"), any());
    }

    @Test
    @DisplayName("Should skip payment when status is not RESERVED")
    void shouldSkipWhenNotReserved() {
        InventoryEvent event = new InventoryEvent(
            "evt-2", 1L, 1L, 42L, 2, 0, "INSUFFICIENT_STOCK");

        paymentConsumer.handleInventoryUpdated(event);

        verify(paymentService, never()).processPayment(anyLong(), any());
    }
}
```

---

9. Security Layer Testing

9.1 JwtUtil Test

```
@DisplayName("JwtUtil Tests")
class JwtUtilTest {

    private JwtUtil jwtUtil;

    @BeforeEach
    void setUp() {
        // Base64-encoded secret (minimum 256-bit for HS256)
        String secret = Base64.getEncoder().encodeToString(
            "MySuperSecureSecretKeyForJWTTokenGeneration2024!".getBytes());
        jwtUtil = new JwtUtil(secret, 3600000L); // 1 hour expiry
    }

    @Test
    @DisplayName("Should generate valid JWT token")
    void shouldGenerateValidToken() {
        String token = jwtUtil.generateToken("testuser", "USER");
        assertNotNull(token);
        assertTrue(token.split("\\.").length == 3); // JWT has 3 parts
    }

    @Test
    @DisplayName("Should extract username from token")
    void shouldExtractUsername() {
        String token = jwtUtil.generateToken("admin", "ADMIN");
        assertEquals("admin", jwtUtil.extractUsername(token));
    }
```

```java
    @Test
    @DisplayName("Should extract role from token")
    void shouldExtractRole() {
        String token = jwtUtil.generateToken("admin", "ADMIN");
        assertEquals("ADMIN", jwtUtil.extractRole(token));
    }

    @Test
    @DisplayName("Should validate non-expired token")
    void shouldValidateNonExpiredToken() {
        String token = jwtUtil.generateToken("user", "USER");
        assertTrue(jwtUtil.isTokenValid(token));
    }

    @Test
    @DisplayName("Should reject expired token")
    void shouldRejectExpiredToken() {
        // Create JwtUtil with 0ms expiry to force immediate expiration
        String secret = Base64.getEncoder().encodeToString(
            "MySuperSecureSecretKeyForJWTTokenGeneration2024!".getBytes());
        JwtUtil shortLivedJwt = new JwtUtil(secret, 1L); // 1ms expiry

        String token = shortLivedJwt.generateToken("user", "USER");
        // Wait for token to expire
        try { Thread.sleep(10); } catch (InterruptedException ignored) {}

        assertFalse(shortLivedJwt.isTokenValid(token));
    }

    @Test
    @DisplayName("Should reject tampered token")
    void shouldRejectTamperedToken() {
        assertFalse(jwtUtil.isTokenValid("invalid.token.here"));
    }
}
```

---

10. Redis & Caching Tests

10.1 ProductService Cache Test (Conceptual)

```java
@SpringBootTest
@ActiveProfiles("test")
@DisplayName("ProductService Cache Integration Tests")
class ProductServiceCacheTest {

    @Autowired private ProductService productService;
    @MockBean private ProductRepository productRepository;
    @MockBean private ProductEventProducer productEventProducer;
```

```java
    @Test
    @DisplayName("Should cache product on first call and return from cache on second")
    void shouldCacheProductOnFirstCall() {
        Product product = new Product();
        product.setId(1L);
        product.setName("Cached Product");

        when(productRepository.findById(1L)).thenReturn(Optional.of(product));

        // First call — hits DB
        productService.getProductById(1L);
        // Second call — should hit cache (DB not called again)
        productService.getProductById(1L);

        // Repository should be called only ONCE (second call served from cache)
        verify(productRepository, times(1)).findById(1L);
    }
}
```

10.2 InventoryService Redis Lock Test

```java
@ExtendWith(MockitoExtension.class)
@DisplayName("InventoryService Tests")
class InventoryServiceTest {

    @Mock private InventoryRepository repository;
    @Mock private ProcessedEventRepository processedEventRepository;
    @Mock private StringRedisTemplate redisTemplate;
    @Mock private ValueOperations<String, String> valueOps;
    @InjectMocks private InventoryService inventoryService;

    @BeforeEach
    void setUp() {
        when(redisTemplate.opsForValue()).thenReturn(valueOps);
    }

    @Test
    @DisplayName("Should reserve stock when lock acquired and stock sufficient")
    void shouldReserveStock() {
        when(valueOps.setIfAbsent(anyString(), anyString(), any()))
            .thenReturn(true);  // Lock acquired
        when(valueOps.get(anyString())).thenReturn(null);

        Inventory inventory = new Inventory(42L, 100);
        when(repository.findByProductId(42L)).thenReturn(Optional.of(inventory));
        when(repository.save(any())).thenReturn(inventory);

        boolean result = inventoryService.reserveStock(42L, 5);

        assertTrue(result);
```

```java
    }

    @Test
    @DisplayName("Should return false when Redis lock not acquired")
    void shouldReturnFalseWhenLockNotAcquired() {
        when(valueOps.setIfAbsent(anyString(), anyString(), any()))
            .thenReturn(false); // Lock NOT acquired

        boolean result = inventoryService.reserveStock(42L, 5);

        assertFalse(result);
        verify(repository, never()).findByProductId(anyLong());
    }

    @Test
    @DisplayName("Should skip duplicate events (idempotency)")
    void shouldSkipDuplicateEvents() {
        when(processedEventRepository.existsByEventId("evt-1")).thenReturn(true);

        inventoryService.restoreStock(42L, 5, "evt-1");

        verify(repository, never()).save(any()); // No DB write on duplicate
    }
}
```

---

11. Mocking Guide

11.1 Mocking Repository Calls

```java
// Return a specific entity
when(orderRepository.findById(1L)).thenReturn(Optional.of(order));

// Return empty (entity not found)
when(orderRepository.findById(999L)).thenReturn(Optional.empty());

// Return saved entity (capture what was saved)
when(orderRepository.save(any(Order.class))).thenAnswer(invocation -> {
    Order saved = invocation.getArgument(0);
    saved.setId(1L); // Simulate auto-generated ID
    return saved;
});

// Return list
when(orderRepository.findAll()).thenReturn(List.of(order1, order2));
```

11.2 Mocking Kafka Producers

```java
// Mock KafkaTemplate.send() to return a completed future
CompletableFuture<SendResult<String, Object>> future =
CompletableFuture.completedFuture(null);
when(kafkaTemplate.send(anyString(), anyString(), any())).thenReturn(future);

// Verify the event was published to the correct topic
verify(kafkaTemplate).send(eq("order-created"), anyString(), any(OrderEvent.class));

// Mock KafkaTemplate.send() to simulate failure
CompletableFuture<SendResult<String, Object>> failedFuture = new CompletableFuture<>();
failedFuture.completeExceptionally(new RuntimeException("Kafka broker unavailable"));
when(kafkaTemplate.send(anyString(), anyString(), any())).thenReturn(failedFuture);
```

11.3 Mocking Redis Operations

```java
@Mock private StringRedisTemplate redisTemplate;
@Mock private ValueOperations<String, String> valueOps;

@BeforeEach
void setUp() {
    when(redisTemplate.opsForValue()).thenReturn(valueOps);
}

// Mock SETNX (distributed lock acquisition)
when(valueOps.setIfAbsent("lock:key", "value", Duration.ofSeconds(30)))
    .thenReturn(true);  // Lock acquired

// Mock GET (check lock ownership)
when(valueOps.get("lock:key")).thenReturn("expected-value");

// Mock DELETE (lock release)
when(redisTemplate.delete("lock:key")).thenReturn(true);
```

11.4 Mocking External API Calls (RestTemplate/WebClient)

```java
@Mock private RestTemplate restTemplate;

// Mock successful API call
when(restTemplate.getForObject("http://product-service/api/products/1", ProductDto.class))
    .thenReturn(new ProductDto(1L, "Laptop", BigDecimal.valueOf(999.99)));

// Mock API failure (timeout)
when(restTemplate.getForObject(anyString(), any()))
    .thenThrow(new RestClientException("Connection timed out"));
```

---

12. Real Production Testing Scenarios

12.1 Scenario Matrix

Scenario	Layer	How to Simulate
Successful order placement	Service	Mock repo.save() to return order
Validation failure (null productId)	Controller	Send invalid JSON → assert 400
Database connection failure	Service	`when(repo.save(any())).thenThrow(new DataAccessException("DB down"))`
Order not found	Service	`when(repo.findById(99L)).thenReturn(Optional.empty())` → assert RuntimeException
Kafka broker unavailable	Producer	Mock `kafkaTemplate.send()` to return failed CompletableFuture
Insufficient inventory	Service	Set `inventory.availableQuantity = 0` → assert `reserveStock()` returns false
Duplicate Kafka message (idempotency)	Service	`when(processedEventRepo.existsByEventId("evt-1")).thenReturn(true)` → verify no DB write
Concurrent stock reservation (Redis lock fail)	Service	Mock `setIfAbsent()` to return false → assert reservation fails
JWT token expired	Security	Create token with 1ms expiry → assert `isTokenValid()` = false
Invalid password at login	Service	Mock `passwordEncoder.matches()` to return false → assert exception
Unauthorized order cancellation	Service	Call `cancelOrder(orderId, differentUserId)` → assert exception
Optimistic lock conflict	Service	Mock `repo.save()` to throw `OptimisticLockingFailureException`
Payment failure (simulated)	Service	Verify both SUCCESS and FAILED paths

12.2 Exception Scenario Example

```java
@Test
@DisplayName("Should handle database failure gracefully")
void shouldHandleDatabaseFailure() {
    when(orderRepository.save(any()))
        .thenThrow(new DataAccessException("Connection refused") {});

    assertThrows(DataAccessException.class,
        () -> orderService.placeOrder(orderRequest, 1L));
```

```
}

@Test
@DisplayName("Should handle Kafka send failure")
void shouldHandleKafkaSendFailure() {
    when(orderRepository.save(any())).thenReturn(savedOrder);
    doThrow(new RuntimeException("Broker not available"))
        .when(orderProducer).sendOrderEvent(any());

    assertThrows(RuntimeException.class,
        () -> orderService.placeOrder(orderRequest, 1L));
}
```

---

13. Test Coverage Strategy

13.1 Coverage Targets

Layer	Target Coverage	Rationale
Service Layer	90%+	Contains all business logic — highest priority
Controller Layer	85%+	Every endpoint must be tested
Repository Layer	70%+	Custom queries need testing; standard CRUD is tested by Spring
Kafka Layer	85%+	Event-driven logic is critical
Security Layer	80%+	Auth/auth bugs = security vulnerabilities
Config Classes	50%+	Only if they contain conditional logic
Overall Project	80%+	Industry standard for enterprise applications

13.2 Classes That MUST Be Tested

- ✅ `OrderService`, `AuthService`, `InventoryService`, `PaymentService`, `ProductService`, `UserService`
- ✅ All Controllers (OrderController, AuthController, etc.)
- ✅ All Kafka Consumers and Producers
- ✅ `JwtUtil` (security-critical)
- ✅ `GlobalExceptionHandler` (via controller tests)
- ✅ Custom repository queries

13.3 When Testing Can Be Skipped

- ❌ DTOs/POJOs with only getters/setters
- ❌ Event classes (data carriers only)
- ❌ `EurekaServerApplication` (infrastructure)
- ❌ Main `Application.java` classes
- ❌ Simple config classes that just declare beans

13.4 JaCoCo Setup (pom.xml)

```xml
<plugin>
    <groupId>org.jacoco</groupId>
    <artifactId>jacoco-maven-plugin</artifactId>
    <version>0.8.11</version>
    <executions>
        <execution>
            <goals><goal>prepare-agent</goal></goals>
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals><goal>report</goal></goals>
        </execution>
        <execution>
            <id>check</id>
            <phase>verify</phase>
            <goals><goal>check</goal></goals>
            <configuration>
                <rules>
                    <rule>
                        <element>BUNDLE</element>
                        <limits>
                            <limit>
                                <counter>LINE</counter>
                                <value>COVEREDRATIO</value>
                                <minimum>0.80</minimum>
                            </limit>
                        </limits>
                    </rule>
                </rules>
            </configuration>
        </execution>
    </executions>
</plugin>
```

Run coverage: `mvn clean verify` → Report at `target/site/jacoco/index.html`

13.5 Maven Surefire Plugin

```xml
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.2.5</version>
    <configuration>
        <includes>
            <include>**/*Test.java</include>
            <include>**/*Tests.java</include>
```

```
        </includes>
    </configuration>
</plugin>
```

Run all tests: `mvn test` **Run specific test**: `mvn test -Dtest=OrderServiceTest`

---

14. Common Testing Mistakes

14.1 Mistake: Testing Framework Instead of Business Logic

```
// ❌ BAD — Tests that Spring Boot saves to the database (framework test)
@Test
void testRepositorySave() {
    Order order = new Order();
    order.setUserId(1L);
    orderRepository.save(order);
    assertNotNull(orderRepository.findById(order.getId())); // Tests JPA, not your code
}

// ✅ GOOD — Tests YOUR business logic using the repository
@Test
void shouldCalculateTotalPriceAndSaveOrder() {
    when(orderRepository.save(any())).thenReturn(savedOrder);
    OrderResponse response = orderService.placeOrder(request, 1L);
    verify(orderRepository).save(argThat(o ->
        o.getTotalPrice().equals(BigDecimal.valueOf(1999.98))));
}
```

14.2 Mistake: Writing Brittle Tests

```
// ❌ BAD — Tests exact string representation (breaks on minor changes)
@Test
void testToString() {
    assertEquals("Order{id=1, userId=1, productId=1, status=CREATED}",
        order.toString());
}

// ✅ GOOD — Tests behavior, not implementation details
@Test
void shouldHaveCreatedStatusAfterPlacement() {
    OrderResponse r = orderService.placeOrder(request, 1L);
    assertEquals("CREATED", r.getStatus());
}
```

14.3 Mistake: Overusing Integration Tests

```java
// ❌ BAD — Full @SpringBootTest to test simple service logic (slow, heavy)
@SpringBootTest
class OrderServiceTest {
    @Autowired OrderService orderService;
    // Loads ENTIRE context including Kafka, Redis, DB...
}

// ✅ GOOD — Unit test with mocks (fast, isolated)
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository orderRepository;
    @InjectMocks OrderService orderService;
}
```

14.4 Mistake: Not Mocking Dependencies

```java
// ❌ BAD — Calls real Kafka producer in unit test
@Test
void testPlaceOrder() {
    // This sends a REAL Kafka message! Fails if Kafka is down.
    orderService.placeOrder(request, 1L);
}

// ✅ GOOD — Mocks the producer
@Test
void testPlaceOrder() {
    doNothing().when(orderProducer).sendOrderEvent(any());
    orderService.placeOrder(request, 1L);
    verify(orderProducer).sendOrderEvent(any()); // Verify it was called
}
```

14.5 Mistake: Not Testing Error Paths

```java
// ❌ BAD — Only tests the happy path
@Test void shouldPlaceOrder() { ... }

// ✅ GOOD — Also tests all error scenarios
@Test void shouldPlaceOrder() { ... }
@Test void shouldThrowWhenProductIdNull() { ... }
@Test void shouldThrowWhenQuantityZero() { ... }
@Test void shouldThrowWhenDatabaseDown() { ... }
@Test void shouldHandleKafkaFailure() { ... }
```

14.6 Summary: Testing Best Practices

Practice	Description

Test behavior, not implementation	Test what a method does, not how it does it
One assertion focus per test	Each test validates one specific behavior
Use descriptive test names	`shouldThrowWhenCancellingPaidOrder()` not `test3()`
Keep tests fast	Unit tests < 100ms each. If slow, you're doing too much
Test edge cases	Null inputs, empty lists, boundary values, concurrent access
Mock external dependencies	DB, Kafka, Redis, REST calls — all must be mocked in unit tests
Use `@Nested` for grouping	Organize by method or feature for readability
Run tests before every commit	`mvn test` should be part of your workflow

---

Document generated from analysis of the Real-Time Order Processing System — 10 microservices, 91 Java source files, covering all enterprise testing patterns.